

Reviewer Pack — Cheeger Constant Lower Bound on Tseitin Resolution Length

Entry 9dd1a19fd8df · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-07 19:11:55 UTC

Cheeger Constant Lower Bound on Tseitin Resolution Length

Entry ID: 9dd1a19fd8df Verdict: INCONCLUSIVE Recorded: 2026-05-07 19:11:55 UTC

1. Conjecture

Field A (mathematical branch): Spectral Graph Theory **Field B** (complexity object): Resolution Proof Length

Statement:

For any connected graph G with n vertices, the resolution proof length of the Tseitin formula derived from G is at least $2^{\Omega(h(G))}$, where $h(G)$ is the Cheeger constant of G .

Rationale (proposer’s reasoning):

The Cheeger constant captures the expansion properties of G , which are known to influence the complexity of Tseitin formulas. A higher Cheeger constant (better expansion) implies longer resolution proofs, as expanders require exponential proof lengths.

Taxonomy category: TSEITIN_RES (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3cfd037ddfaccb36

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_graph(n):
        if n <= 1:
            return []
        edges = set()
        for i in range(1, n):
            j = random.randint(0, i-1)
            edges.add((j, i))
        return list(edges)

    def cheeger_constant(graph, n):
        min_cut = float('inf')
        for vertex in range(n):
            neighbors = [neighbor for neighbor, _ in graph if neighbor != vertex]
            cut_size = len(neighbors)
            boundary_size = sum(1 for _, neighbor in graph if neighbor == vertex and neighbor not
                               if boundary_size > 0:
                                   min_cut = min(min_cut, cut_size / boundary_size)
        return min_cut

    def tseitin_formula(graph, n):
        clauses = []
        for edge in graph:
            u, v = edge
            clause = [f"p{u}", f"p{v}"]
            clauses.append(clause)
            clauses.append([f"-p{u}", f"-p{v}"])
            clauses.append([f"-p{u}", f"p{v}"])
            clauses.append([f"p{u}", f"-p{v}"])
        return clauses

    def dpll_solve(clauses):
        def dpll(model, clauses):
            if not clauses:
                return True
            literal = next(l for l in model.keys() if model[l] is None)
            for value in [True, False]:
                new_model = {**model, literal: value}
                new_clauses = []
                for clause in clauses:
```

```

        if any(new_model.get(l) == (not v) for l, v in zip(clause, [True, False])):
            continue
        new_clause = [l for l, v in zip(clause, [True, False]) if new_model[l] != v]
        if not new_clause:
            return False
        new_clauses.append(new_clause)
    if dpll(new_model, new_clauses):
        return True
    return False

model = {f"p{i}": None for i in range(n)}
return dpll(model, clauses)

n = random.randint(5, 40)
graph = generate_random_graph(n)
h_G = cheeger_constant(graph, n)
tseitin_clauses = tseitin_formula(graph, n)
proof_length = len(tseitin_clauses) if dpll_solve(tseitin_clauses) else float('inf')

return {
    "metric_name": "resolution_proof_length",
    "metric_value": proof_length,
    "instances_tested": 1,
    "conjecture_holds": proof_length >= 2 ** (math.ceil(math.log2(h_G)) if h_G > 0 else 0),
    "counterexample": "" if conjecture_holds else f"Graph with n={n}, h(G)={h_G} has proof length {proof_length}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={results[0]['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2c326aad.py", line 93, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2c326aad.py", line 77, in run_trial
    proof_length = len(tseitin_clauses) if dpll_solve(tseitin_clauses) else float('inf')
                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2c326aad.py", line 71, in dpll_solve
    return dpll(model, clauses)
           ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2c326aad.py", line 62, in dpll
    new_clause = [l for l, v in zip(clause, [True, False]) if new_model[l] != v]
                                     ~~~~~~^
KeyError: '-p24'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before producing data, preventing evaluation of conjecture support or falsification. | next: Debug and rerun the test with error handling for clause variables

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	104662
2	propose	ollama_remote	qwen3:8b	0	0	35488
3	preregistration	ollama_remote	qwen3:8b	0	0	19862
4	novelty	ollama_remote	qwen3:8b	0	0	16428
5	novelty	ollama_remote	qwen3:8b	0	0	11348
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19741
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10728
8	judge	ollama_remote	qwen3:8b	0	0	10721

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 228979 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/9dd1a19fd8df.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71

2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9dd1a19fd8df.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9dd1a19fd8df.tar.gz` (if generated)